Python语言与系统设计

（Python序列）

Python Language & System Design

第3章 程序的控制结构

(Control Structures of Program)

合肥工业大学 计算机与信息学院



第3章 程序的控制结构



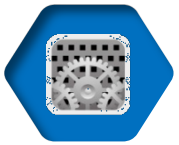
3.1 程序的基本结构



3.2 分支结构



3.3 循环结构



3.4 案例



3.5 本章小结



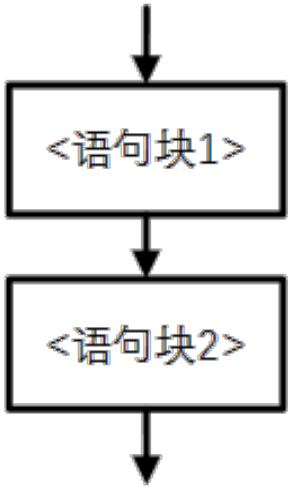
3.1 程序的基本结构

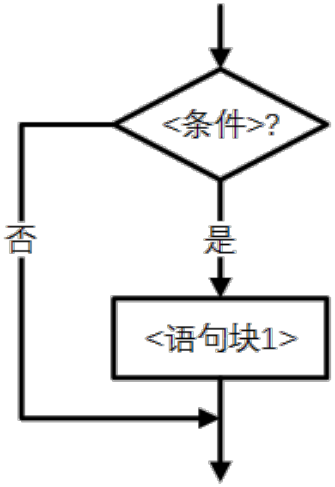
程序的基本结构概述

- 顺序结构是程序的基础，但单一的顺序结构不可能解决所有问题。
- 程序(Program)由三种基本结构组成：
  - 顺序结构(**Sequence Structure**)
  - 分支结构(**Branch Structure**)
  - 循环结构(**Loop Structure**)

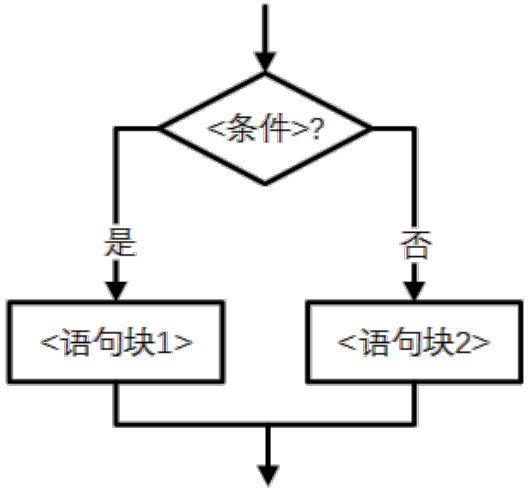
3.1 程序的基本结构

- 顺序结构：程序按照线性顺序依次执行的一种运行方式，如：语句块1和语句块2表示一个或一组顺序执行的语句
- 分支结构：程序根据条件判断结果而选择不同向前执行路径的一种运行方式，包括单分支结构和二分支结构(二分支结构会组合形成多分支结构)



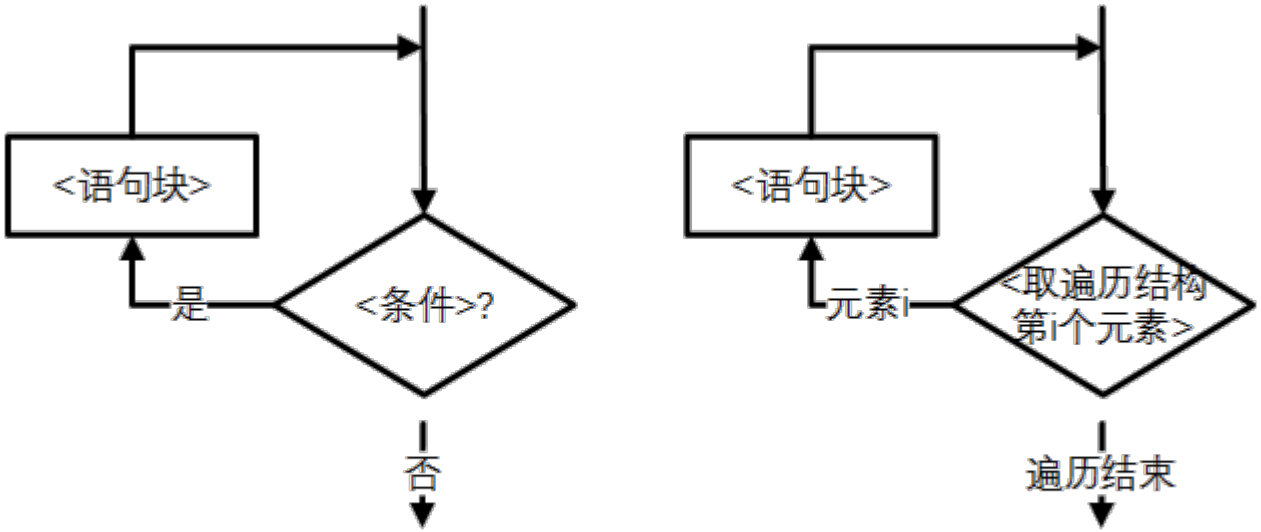


单分支结构



二分支结构

■ 循环结构：程序根据条件判断结果向后反复执行的一种运行方式，根据循环体触发条件不同，包括条件循环和遍历循环结构



• 循环结构示例

条件表达式(Conditional Expression)

- 在选择和循环结构中，都要使用条件表达式来确定下一步的执行流程。
- 条件表达式的值只要不是False、0（或0.0、0j等）、空值None、空列表、空元组、空集合、空字典、空字符串、空range对象或其他空迭代对象，Python解释器均认为与True等价。

表达式中可以使用的运算符：

- 算术运算符：+、-、*、/、//、%、**
- 关系运算符：>、<、==、<=、>=、!=
- 测试运算符：in、not in、is、is not
- 逻辑运算符：and、or、not
- 位运算符：~、&、|、 ^、 <<、>>
- 矩阵乘法运算符：@



3.1 程序的基本结构

```
>>> if 3:  #使用整数作为条件表达式
    print(5)
5

>>> a = [1, 2, 3]
>>> if a:  #使用列表作为条件表达式
    print(a)
[1, 2, 3]

>>> a = []
>>> if a:
    print(a)
else:
    print('empty')
Empty

>>> s = 0
>>> for i in range(0, 11, 1):  #遍历迭代对象中的所有元素
    s += i
>>> print(s)
55
```



3.1 程序的基本结构

- 逻辑运算符and和or以及关系运算符具有惰性求值特点，只计算必须计算的表达式。

✓ 例如：

```
>>> 3>5 and a>3     #注意，此时并没有定义变量a
False
>>> 3<5 or a>3      #3<5的值为True，不需要计算后面表达式
True
```

- 条件表达式中不允许使用赋值运算符“=”。

```
>>> if a=3:
```

异常

3.1 程序的基本结构

✓ 天天向上实例

■ 一年365天，以第1天的能力值为基数，记为1.0，当好好学习时能力值相比前一天提高r‰，当没有学习时由于遗忘等原因能力值相比前一天下降r‰。每天努力和每天放任，一年下来的能力值相差多少呢？

```
import math
r = eval(input("ratio:"))
dayup = math.pow((1.0 + r), 365)
daydown = math.pow((1.0 - r), 365)
print("向上:{:.2f}, 向下:{:.2f}.".format(dayup, daydown))
```

r = 0.001，每天努力1‰，一年下来将提高44%；
r = 0.005，每天努力5‰，一年下来将提高6倍！
r = 0.01，每天努力1%，一年下来将提高37倍！



■ 这就是天天向上的力量！

✓ 天天向上实例2

■ 一年365天，一周5个工作日，如果每个工作日都很努力，可以提高r%，仅在周末放任一下，能力值每天下降r%，效果如何呢？

```
dayup, r = 1.0, 0.01
for i in range(365):
    if i % 7 in [6, 0]:#周六周日
        dayup = dayup * (1 - r)
    else:
        dayup = dayup * (1 + r)
print("向上5天向下2天的力量: {:.2f}.".format(dayup))
```

r = 0.01，每周努力5天，而不是每天，一年下来，水平仅是初始的4.63倍！



每周工作5天，休息2天，休息日水平下降0.01，工作日要努力到什么程度一年后的水平才与每天努力1%所取得的效果一样呢？

3.2 分支结构 (选择结构）

单分支结构

- if <表达式>:
  <语句块>

```
x = input('Input two number:')
a, b = map(int, x.split())
if a > b:
    a, b = b, a
print(a, b)
```

```
Input two number:9 2
2 9
>>>
```

双分支结构

- if <表达式>:
  <语句块1>
  else:
  <语句块2>

```
x = eval(input("请输入一个实数"))
if x < 0:
    print(-x)
else:
    print(x)
```

3.2 分支结构

双分支结构

■双分支结构的另一种更简洁的表达方式，适合通过判断返回特定值，语法格式如下：

<表达式1> if <条件> else <表达式2>

```
>>> a = 5
>>> print(6) if a>3 else print(5)

6
```

这个结构的表达式惰性求值，以下表示式没有import math，不会报错

```
>>> import random
>>> x = math.sqrt(9) if 2>3 else random.randint(1, 100)
>>> x
31
```

3.2 分支结构

- 多分支结构

if-elif-else描述多分支结构，语句格式如下：

```
if <条件1>:
    <语句块1>
elif <条件2>:
    <语句块2>
...
else:
    <语句块N>
```

3.2 分支结构

- 多分支结构是二分支结构的扩展
- 如果没有任何条件成立，else语句块被执行。else子句是可选的

```
PM = eval(input("请输入PM2.5数值: "))
if 0<= PM < 35:
    print("空气优质，快去户外运动!")
elif 35 <= PM <75:
    print("空气良好，适度户外活动！")
else:
    print("空气污染，请小心！")
```

3.2 分支结构

➢ 选择语句的嵌套

```
if <表达式1>:
    <语句块1>
    if <表达式2>:
        <语句块2>
    else:
        <语句块3>
else:
    if <表达式4>:
        <语句块4>
```

注意：缩进必须要正确并且一致。

```
a=int(input("请输入一个正整数:"))
if a%2==0:
    if a%3==0:
        print(a,"能同时被2和3整除")
    else:
        print(a,"能被2，但不能被3整除")
else:
    print(a,"不能被2整除")
```

3.2 分支结构

分支结构的应用

- 例：编写程序，判断今天是今年的第几天？

```
import time
date = time.localtime()                    #获取当前日期时间
year, month, day = date[:3]
day_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):
                                           #判断是否为闰年
    day_month[1] = 29                      #闰年2月为29天
if month==1:
    print(day)
else:
    print(sum(day_month[:month-1])+day)
```

```
>>> import time
>>> date = time.localtime()
>>> date
time.struct_time(tm_year=2020, tm_mon=10, tm_mday=8, tm_hour=15, tm_min=3, tm_se
c=48, tm_wday=3, tm_yday=282, tm_isdst=0)
```

3.2 分支结构

- 或者使用标准函数库**datetime**直接计算今天是今年的第几天
- **datetime**库以类的方式提供多种日期和时间表达方式
  - datetime.date：日期表示类，可以表示年、月、日等
  - datetime.time：时间表示类，可以表示小时、分钟、秒、毫秒等
  - datetime.datetime：日期和时间表示的类，功能覆盖date和 time类
  - datetime.timedelta：时间间隔有关的类
  - datetime.tzinfo：与时区有关的信息表示类

```
>>> import datetime
>>> datetime.datetime.now()
datetime.datetime(2020, 10, 8, 15, 10, 28, 581300)

>>> datetime.date.today().timetuple().tm_yday
282
>>> datetime.date(2021,5,12).timetuple().tm_yday
132
```

3.3 循环结构

- Python提供了两种基本的循环结构语句——while和for。
- 相同或不同的循环结构之间都可以互相嵌套，实现更为复杂的逻辑。
- for循环和while循环都可以带else, 如果循环是因为break结束的，就不执行else中的代码。

```
while <条件表达式>:
    <循环体>
[else:
    <else子句代码块>]
```

```
for <取值 in 序列或迭代对象>:
    <循环体>
[else:
   <else子句代码块>]
```

3.3 循环结构

➢ 循环结构的优化

- 为了优化程序以获得更高的效率和运行速度，在编写循环语句时，应尽量减少循环内部不必要的计算，将与循环变量无关的代码尽可能地提取到循环之外。
- 多重循环嵌套时，减少内层不必要的计算

```
import time
digits = (1, 2, 3, 4)
start=time.time()
for i in range(1000):
    result = []
    for i in digits:
        for j in digits:
            for k in digits:
                result.append(i*100+j*10+k)
print(time.time()-start)
```

优化后

```
for i in range(1000):
    result = []
    for i in digits:
        i=i*100
        for j in digits:
            j=j*10
            for k in digits:
                result.append(i+j+k)
```

- 在循环中应尽量引用局部变量，因为局部变量的查询和访问速度比全局变量略块。

3.3 循环结构

➢ break和continue语句

- break语句被执行，将使得整个循环提前结束。
- continue语句的作用是终止当前循环，并忽略continue之后的语句，然后回到循环的顶端，提前进入下一次循环。

✓ 计算小于100的最大素数，注意break语句和else子句的用法。

```
for n in range(100, 1, -1):
    for i in range(2, n):
        if n % i == 0:
            break
    else:
        print(n)
        break
```

```
97
```



输出100以内的所有素数，怎么修订?

删除最后一个break

3.3 循环结构

输出10以内的奇数

```
i=0
while i<10:
    if i%2==0:
        continue
    print(i)
    i+=1
```

- 警惕continue带来的问题：死循环,Ctrl+C强行终止。



怎么修订?

```
for i in range(10):
    if i%2==0:
        continue
    print(i, end=' ')
```

```
1 3 5 7 9
```

课堂练习

```
for s in "Hello Python":
    if s=="o":
        continue
    print(s, end="")
else:
    print("exit")
```

输出结果：Hell Pythnexit

3.4 案例

✓ 例：计算1+2+3+…+100 的值。

```
#解法一：
s=0
for i in range(1,101):
    s = s + i
print('1+2+3+...+100 = ', s)
```

```
#解法二：
print('1+2+3+...+100 = ', sum(range(1,101)))
```

✓ 例：输出序列中的元素。

遍历一般用for语句

```
a_list=['a', 'b', 'c', 'd', 'e']
for i, v in enumerate(a_list):
    print('列表的第', i+1, '个元素是：', v)
```

```
列表的第 1 个元素是： a
列表的第 2 个元素是： b
列表的第 3 个元素是： c
列表的第 4 个元素是： d
列表的第 5 个元素是： e
>>>
```

3.4 案例

- 例：输出“水仙花数”。所谓水仙花数是指1个3位的十进制数，其各位数字的立方和等于该数本身。例如：153是水仙花数，因为153 = 1^3 + 5^3 + 3^3 。

```
#解法一：
for i in range(100, 1000):
    bai, shi, ge = map(int, str(i))
    if ge**3 + shi**3 + bai**3 == i:
        print(i)
```

```
======:
153
370
371
407
>>>
```

```
#解法二：
for num in range(100, 1000):
    r = map(lambda x:int(x)**3, str(num))
    if sum(r) == num:
        print(num)
```

3.4 案例

- 例：求[70, 90, 78, 85, 97, 94, 65, 80]的平均分。

```
score = [70, 90, 78, 85, 97, 94, 65, 80]
s = 0
for i in score:
    s += i
print(s/len(score))
```

```
score = [70, 90, 78, 85, 97, 94, 65, 80]
print(sum(score) / len(score))
```

- 例：判断一个数是否为素数。

```
import math
n = input('Input an integer:')
n = int(n)
m = math.ceil(math.sqrt(n)+1)
for i in range(2, m):
    if n%i == 0 and i<n:
        print('No')
        break
else:
    print('Yes')
```

3.4 案例

- 例：编写程序，生成一个含有20个随机数的列表，要求所有元素不相同，并且每个元素的值介于1到100之间。

```
import random

x = []
while True:
    if len(x)==20:
        break
    n = random.randint(1, 100)
    if n not in x:
        x.append(n)
print(x)
print(len(x))
```

- 用集合更简单

```
from random import randint

x = set()
while len(x)<20:
    x.add(randint(1,100))
print(x)
print(len(x))
```

```
{3, 8, 10, 15, 19, 23, 33, 40, 53, 60, 72, 73, 77, 80, 81, 90, 92, 95, 96, 99}
20
>>>
```

3.4 案例

✓ **例 编写程序，计算组合数C(n,i)，即从n个元素中任选i个，有多少种选法。**

根据组合数定义，需要计算3个数的阶乘，在很多编程语言中都很难直接使用整型变量表示大数的阶乘 结果，虽然Python并不存在这个问题，但是计算大数的阶乘仍需要相当多的时间。以Cni(8,3)为例，

$$C_8^3 = \frac{8!}{3!\times(8-3)!} = \frac{8\times7\times6\times5\times4\times3\times2\times1}{3\times2\times1\times5\times4\times3\times2\times1}$$

```
n, i = eval(input("n:")),eval(input("i:"))
if not (isinstance(n,int) and isinstance(i,int) and n>=i):
    print('n and i must be integers and n >= i.')
    return
result = 1
Min, Max = sorted((i,n-i))
for i in range(n,0,-1):
    if i>Max:
        result *= i
    elif i<=Min:
        result //= i
print(result)
```

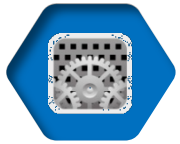
3.4 案例

$$C_8^3 = \frac{8!}{3!\times(8-3)!} = \frac{8\times7\times6\times5\times4\times3\times2\times1}{3\times2\times1\times5\times4\times3\times2\times1}$$

```
n, i = eval(input("n:")),eval(input("i:"))
minNI = min(i, n-i)
result = 1
for j in range(0, minNI):
    result = result * (n-j) // (j+1)
print(result)
```



此段代码与前一段代码相比，效率有提高吗？



3.4 案例

- 也可以使用math库中的阶乘函数直接按组合数定义实现。

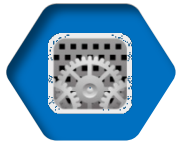
```
>>>import math
>>>n, i = eval(input("n:")),eval(input("i:"))
n: 6
i: 2
>>>print(int(math.factorial(n)/math.factorial(i)/math.factorial(n-i)))
15
```

- 还可以直接使用Python标准库itertools提供的函数。

```
>>> tuple(itertools.combinations(range(4),2))
((0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3))
>>>
```

创建一个迭代器，返回iterable中所有长度为r的子序列

```
>>> import itertools
>>> len(tuple(itertools.combinations(range(60),2)))
1770
```



3.4 案例

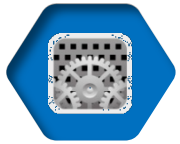
- **itertools还提供了排列函数permutations()**

```
>>> import itertools
>>> for item in itertools.permutations(range(1,4),2):
    print(item)
```

```
(1, 2)
(1, 3)
(2, 1)
(2, 3)
(3, 1)
(3, 2)
```

- **itertools还提供了根据一个序列的值 对 另一个序列进行过滤的函数compress()**

```
>>> import itertools
>>> x = range(1, 20)
>>> y = (1,0)*9+(1,)
>>> y
(1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1)  #0表示False
>>> list(itertools.compress(x, y))
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```



3.4 案例

- itertools还提供了用于循环遍历可迭代对象元素的函数cycle()

```
>>> import itertools
>>> x = 'Private Key'
>>> y = itertools.cycle(x) #循环遍历序列中的元素
>>> for i in range(20):
    print(next(y), end=',')
P,r,i,v,a,t,e, ,K,e,y,P,r,i,v,a,t,e, ,K,
>>> for i in range(5):
    print(next(y), end=',')
e,y,P,r,i,
```

3.4 案例

✓ 例：编写程序，计算百钱买百鸡问题。



假设公鸡5元一只，母鸡3元一只，小鸡1元三只，现在有100块钱，想买100只鸡，问有多少种买法？

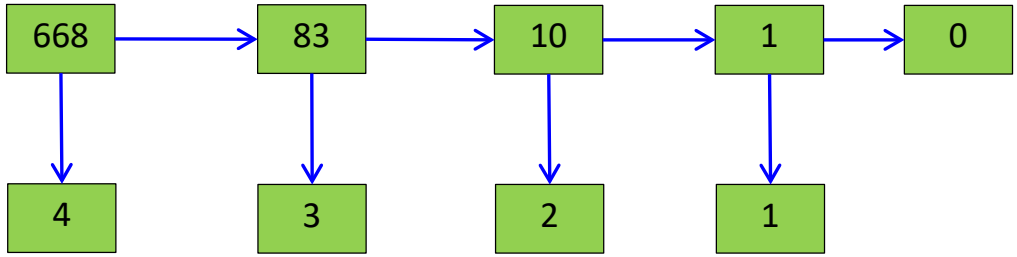
```
#假设能买x只公鸡
for x in range(21):
    #假设能买y只母鸡
    for y in range(34):
        #假设能买z只小鸡
        z = 100-x-y
        if z%3==0 and 5*x + 3*y + z//3 == 100:
            print(x,y,z)
```

```
0 25 75
4 18 78
8 11 81
12 4 84
```

3.4 案例

✓ 例 编写程序，实现十进制整数到其他任意进制的转换。

- 编程要点：除基取余，逆序排列
- 十进制数668转换为八进制数字的过程如下图所示，其中横着向右的箭头表示左边的数字除以8得到的商，而向下的箭头表示上面的数字除以8得到的余数。当商为0时，算法结束，最后把得到的余数4321逆序得到1234。




```
n, base = eval(input("n:")),eval(input("base:"))
result = []
div = n
while div != 0:
    div, mod = divmod(div, base)
    result.append(mod)
result.reverse()
result = ''.join(map(str, result))
print(eval(result))
```

3.4 案例

✓ 例 计算前n个自然数的阶乘之和1!+2!+3!+...+n!的值。

```
n = evl(input("n:"))
 result, t = 1, 1
 for i in range(2, n+1):
     t *= i
     result += t
```

```
>>> from math import factorial
>>> sum(map(factorial, range(1, 100)))
```

这一方法效率会低一些！Add why?

```
9427862397658265791605952682068393813547543496010509743453954104070782302495904144588301174426181807329112035202088937164165912135655644233652892042094031 3
```

3.4 案例

- 例 判断一个数字是否为丑数。
  - 一个数的因数如果只包含2、3、5，那么这个数是丑数。

```
n = eval(input("n:"))
for i in (2, 3, 5):
    while True:
        m, r = divmod(n, i)
        if r != 0:
            break
        else:
            print(m)
            n = m
print(n==1)
```

```
=========
n:15
5
1
True
>>>
=========
n:14
7
False
>>>
```

3.4 案例

✓ 例 检测序列元素是否满足严格升序关系。

```
tests = ('abcdeff', [1,2,3,5,4], (3,5,7,9))
for seq in tests:
    for index, value in enumerate(seq[:-1]):
        if value >= seq[index+1]:
            print("false")
    print("true")
```

？？？真的可以么

```
=========
false
true
false
true
true
```

3-4 P47.py - C:/PythonTest/3-4 P47.py (3.9.0)

File Edit Format Run Options Window Help

```
tests = ('abcdeff', [1,2,3,5,4], (3,5,7,9))
for seq in tests:
    flag=True
    for index, value in enumerate(seq[:-1]):
        if value >= seq[index+1]:
            flag=False
    print(flag)
```

Python 3.9.0 Shell

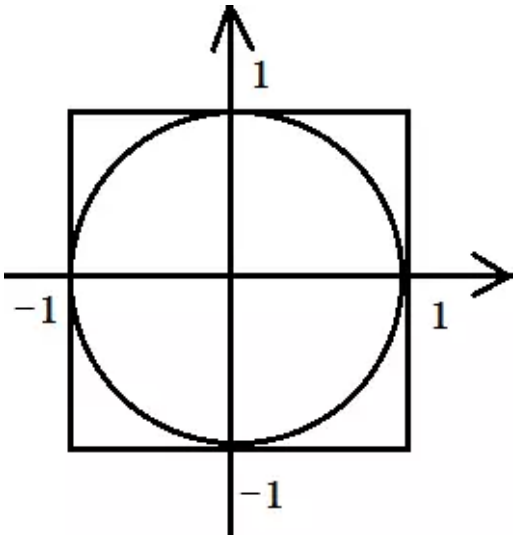
File Edit Shell Debug Options Window Help

```
========== RESTART: C:/PythonTest/3-4 P47.py
False
False
True
>>>
```

3.4 案例

✓ 例：利用蒙特.卡罗方法计算圆周率近似值。

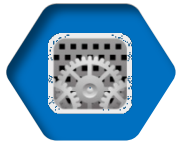
```
from random import random
times = eval(input("times"))
hits = 0
for i in range(times):
    x = random()
    y = random()
    if x*x + y*y <= 1:
        hits += 1
print(4.0 * hits/times)
```




```
times: 10000
times: 1000000
times: 100000000
times: 1000000000
```



```
3.1396
3.1419
3.141688
3.141591436
```



3.4 案例

✓ 例 6174猜想：

1955年，卡普耶卡(D.R.Kaprekar)对4位数字进行了研究，发现一个规律：对任意各位数字不相同的4位数，使用各位数字能组成的最大数减去能组成的最小数，对得到的差重复这个操作，最终会得到6174这个数字，并且这个操作最多不会超过7次。

```
from string import digits
from itertools import combinations
for item in combinations(digits, 4):
    times = 0
    while True:
        big = int(''.join(sorted(item, reverse=True)))
        little = int(''.join(sorted(item)))
        difference = big-little
        times = times+1
        if difference==6174:
            if times>7:
                print(times)
            break
        else:
            item = str(difference)
```

3.5 本章小结

内容回顾

- 程序的基本结构：顺序、分支、循环
- 程序基本结构的应用



思考

- 三种程序基本结构的区别
- break与continue的区别